

CURSO DESENVOLVIMENTO FULL STACK

MÓDULO 10 - TESTE E QUALIDADE DE SOFTWARES

Professor: Bruno Ramalho dos santos

Acesse aqui nossas
redes sociais.



Capítulo 1 : Introdução à Qualidade de Software	2
1. Conceitos de Qualidade de Software	2
2. Importância da Qualidade de Software no Ciclo de Vida de Desenvolvimento (SDLC)	2
3. Modelos de Qualidade de Software: ISO 9126 e ISO 25000 (SQuaRE)	3
4. Processos de Garantia da Qualidade de Software (QA vs. QC)	4
Exercícios de Fixação	4
Capítulo 2 — Fundamentos e Anatomia dos Testes de Software	5
1. Conceitos Básicos de Testes: Teste, Defeito, Falha e Erro	5
2. Níveis de Testes	5
3. Tipos de Testes	6
4. Técnicas de Projeto de Testes	6
Exercícios de Fixação — Dia 2	6
Capítulo 3 — Testes de Unidade e Integração na Prática	8
1. Testes de Unidade: Objetivos e Benefícios	8
Objetivos Principais:	8
Benefícios para o Desenvolvedor Full Stack:	8
2. Ferramentas do Ecossistema de Desenvolvimento	8
3. Exemplo Prático de Caixa Branca (Testando Fluxos Lógicos)	9
O Código da Função (Exemplo em JavaScript/Node.js):	9
A Suíte de Testes de Unidade (Caixa Branca):	9
4. Mocks e Stubs: Isolando Componentes	10
Exercícios de Fixação — Dia 3	11
Capítulo 4 — Integração, Regressão e Validação do Sistema	12
1. Testes de Integração: Objetivos e Abordagens	12
Objetivos dos Testes de Integração:	12
Principais Abordagens de Integração:	12
2. Testes de Regressão: Blindando o Código Existente	12
O que são e qual a sua importância?	12
3. Testes de Sistema: A Visão de Ponta a Ponta (End-to-End)	13
Exercícios de Fixação — Dia 4	13
Capítulo 5 — Automação de Testes, Experiência do Usuário e CI/CD	14
1. Introdução à Automação de Testes: Benefícios e Desafios	14
2. Testes de Interface de Usuário (UI) e Ferramentas do Mercado	14
3. Testes de API: Garantindo o Contrato de Dados	14
Exemplo Prático de Script de Teste de API (Exemplo Conceitual com Cypress):	15
4. Testes de Aceitação, Usabilidade e a Integração Contínua (CI/CD)	15
Exercícios de Fixação — Dia 5	15
REFERÊNCIAS	17

Capítulo 1 : Introdução à Qualidade de Software

1. Conceitos de Qualidade de Software

Historicamente, o sucesso no desenvolvimento de software era medido apenas pela entrega das funcionalidades dentro do prazo. No cenário atual de desenvolvimento Full Stack, a definição de sucesso mudou. Não basta que o sistema "funcione"; ele precisa funcionar de forma consistente, escalável e segura.

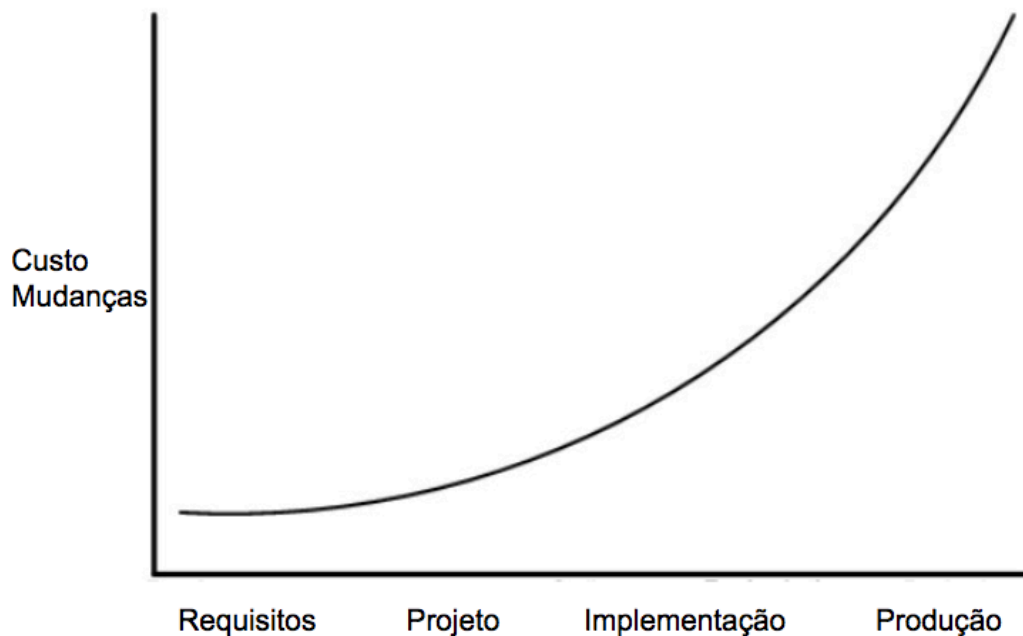
A qualidade de software não é um evento que ocorre no final do projeto, mas sim um atributo intrínseco construído em cada linha de código, commit e escolha arquitetural. Para estruturar esse conceito, analisamos quatro pilares fundamentais:

- **Confiabilidade:** É a capacidade do software de manter seu nível de desempenho sob condições estabelecidas por um período de tempo determinado. Para um desenvolvedor Full Stack, isso significa garantir que a API não caia sob carga moderada e que o banco de dados trate transações de forma íntegra, evitando estados inconsistentes no sistema.
- **Usabilidade:** Refere-se à facilidade com que os usuários finais podem interagir com a aplicação. Um front-end intuitivo, que respeita padrões de acessibilidade, possui tempo de resposta rápido e oferece fluxos de navegação claros, determina o sucesso da experiência do usuário (UX).
- **Eficiência:** Está diretamente ligada à otimização de recursos. O software deve apresentar um tempo de resposta aceitável e consumir uma quantidade razoável de memória e processamento. No front-end, isso reflete no tamanho do *bundle* JavaScript carregado; no back-end, na velocidade das consultas SQL e no consumo de CPU do servidor.
- **Manutenibilidade:** É a facilidade com que o código pode ser modificado para corrigir defeitos, melhorar o desempenho ou adaptar o sistema a novas regras de negócio. Um código com alta manutenibilidade possui baixo acoplamento, alta coesão e é adequadamente coberto por testes.

2. Importância da Qualidade de Software no Ciclo de Vida de Desenvolvimento (SDLC)

Ignorar a qualidade durante as fases iniciais do ciclo de desenvolvimento gera um fenômeno conhecido na engenharia de software como **Débito Técnico**. À medida que o código avança sem os devidos cuidados e testes, o custo para corrigir um erro aumenta exponencialmente.

Custo de Correção:



Quando um defeito é identificado na fase de levantamento de requisitos ou desenho da arquitetura, sua correção custa apenas o tempo de reescrever um documento. Se o mesmo erro chega ao ambiente de produção, ele pode envolver vazamento de dados, insatisfação de clientes, horas de *troubleshooting* da equipe de engenharia e a necessidade de realizar *rollbacks* emergenciais sob pressão.

Portanto, a garantia da qualidade deve ser um processo contínuo e preventivo, e não apenas uma etapa de validação antes do deploy.

3. Modelos de Qualidade de Software: ISO 9126 e ISO 25000 (SQuaRE)

Para que os times de tecnologia possam mensurar a qualidade de forma objetiva, foram criados padrões internacionais normativos:

- **ISO 9126:** Foi o modelo pioneiro que dividiu a qualidade do software em características internas e externas (como Funcionalidade, Confiabilidade, Usabilidade, Eficiência, Automação/Manutenibilidade e Portabilidade). Embora tenha sido uma norma fundamental, ela se tornou obsoleta com a rápida evolução da arquitetura web e de microsserviços.
- **ISO 25000 (SQuaRE - *Software Product Quality Requirements and Evaluation*):** É a evolução direta da ISO 9126. Trata-se de uma família de normas guia que organiza a avaliação da qualidade de forma muito mais robusta, adaptada para softwares modernos. Ela estabelece critérios rígidos para a avaliação

do produto de software, dividindo-se em subcaracterísticas como adequação funcional, compatibilidade, segurança e manutenibilidade.

Para um desenvolvedor Full Stack, a ISO 25000 serve como um guia para mapear requisitos não funcionais. Por exemplo, ao desenhar uma API, o critério de **Segurança** (Criptografia, Autenticação) e **Compatibilidade** (capacidade de interagir com outros sistemas via JSON/REST) são extraídos diretamente dessas diretrizes.

4. Processos de Garantia da Qualidade de Software (QA vs. QC)

Dentro da engenharia de software, existem duas frentes que atuam em conjunto, mas possuem focos distintos:

- **Quality Assurance (QA - Garantia da Qualidade):** É um processo **preventivo** focado nos *processos* de desenvolvimento. O objetivo do QA é garantir que a equipe esteja utilizando as melhores práticas, metodologias corretas e ferramentas adequadas para evitar que os erros aconteçam. (Exemplo: definição de padrões de código, revisões de escopo, implementação de pipelines automatizados).
- **Quality Control (QC - Controle de Qualidade):** É um processo **corretivo** focado no *produto*. O objetivo do QC é encontrar defeitos no software após ele ter sido construído, antes de ser entregue ao cliente. (Exemplo: a execução manual ou automatizada de testes na aplicação pronta).

Em equipes modernas de alta performance (DevOps/Agile), o desenvolvedor Full Stack atua ativamente em ambas as frentes: participando do QA ao estruturar o código seguindo boas práticas e arquiteturas limpas, e participando do QC ao escrever suítes de testes automatizados.

Exercícios de Fixação

1. Explique, com suas próprias palavras, a diferença entre o foco de *Quality Assurance* (QA) e *Quality Control* (QC) dentro de um time de desenvolvimento.
2. Imagine que você desenvolveu um e-commerce que funciona perfeitamente quando acessado por 10 usuários simultâneos, mas cai imediatamente quando recebe 500 acessos no horário do almoço. Baseado nos pilares de qualidade estudados, qual pilar foi violado nessa situação? Justifique.
3. Por que o custo de correção de um bug na fase de produção é severamente maior do que se ele tivesse sido detectado na fase de codificação/desenvolvimento?

Capítulo 2 — Fundamentos e Anatomia dos Testes de Software

1. Conceitos Básicos de Testes: Teste, Defeito, Falha e Erro

Na engenharia de software, palavras como "erro" e "falha" não são sinônimos. Existe uma cadeia causal muito clara que liga uma ação humana a um problema em produção. Compreender essa nomenclatura é fundamental para a comunicação em times de alta performance.

- **Erro (ou Engano):** É uma ação humana que produz um resultado incorreto. Ocorre na mente do desenvolvedor, analista ou designer.
 - *Exemplo:* O desenvolvedor interpreta errado a regra de negócio e esquece que usuários menores de 18 anos não podem comprar um determinado produto.
- **Defeito (ou Bug / Fault):** É a manifestação física do erro no código-fonte. O erro humano gera um defeito escrito no software.
 - *Exemplo:* No código da API, a validação fica condicionalmente errada: `if (idade > 16)` em vez de `if (idade >= 18)`. O defeito está estático no arquivo de código.
- **Falha (Failure):** É o comportamento incorreto do sistema em tempo de execução, ou seja, quando o software deixa de realizar a função exigida. A falha é o defeito sendo executado pelo usuário ou por um teste.
 - *Exemplo:* Um jovem de 17 anos entra no site e consegue finalizar a compra de um produto restrito. O sistema falhou.

💡 **Nota do Professor:** Nem todo defeito resulta em uma falha. Se um trecho de código com defeito nunca for executado (um "código morto"), o sistema nunca apresentará uma falha. Por isso testamos caminhos isolados.

2. Níveis de Testes

Para não tentar testar tudo de uma vez de forma caótica, dividimos os testes em **níveis estratégicos**, que acompanham o amadurecimento do desenvolvimento:

+-----+
|Teste de Aceitação| <- Foco no Cliente/Negócio
+-----+
|Teste de Sistema| <- Foco no Todo (Fim a Fim)
+-----+
|Teste de Integração| <- Foco na Comunicação (Módulos)
+-----+

[Teste de Unidade] <- Foco na Função/Classe

+-----+

- **Teste de Unidade (Unit Test):** Foca na menor parte testável de um software de forma isolada (uma função, um método ou uma classe). Aqui, isolamos o banco de dados e as APIs externas.
- **Teste de Integração:** Verifica a interação e a comunicação entre duas ou mais unidades, módulos ou sistemas externos (ex: validar se a sua API consegue salvar e recuperar dados corretamente do banco de dados PostgreSQL).
- **Teste de Sistema (End-to-End - E2E):** Testa o sistema de ponta a ponta em um ambiente que simula a produção. O objetivo é garantir que todo o fluxo (Front-end, Back-end, Banco de Dados, Mensageria) funcione harmonicamente.
- **Teste de Aceitação:** Geralmente realizado pelo cliente ou pela equipe de Product Owners (PO) para validar se o software atende aos critérios de aceite do negócio e se está pronto para o deploy final.

3. Tipos de Testes

Enquanto os *níveis* dizem respeito a *quando/onde* testamos, os *tipos* dizem respeito a *o que* estamos testando.

- **Teste Funcional:** Avalia o que o software faz baseado nos requisitos funcionais e regras de negócio. (Exemplo: "O sistema deve calcular o desconto de 10% para pagamentos via PIX").
- **Teste Não Funcional:** Avalia como o software faz, focando em aspectos de infraestrutura, segurança, carga e usabilidade. (Exemplo: "A página deve carregar em menos de 2 segundos sob uma carga de 1000 acessos simultâneos").
- **Teste de Regressão:** Consiste em reexecutar testes antigos após uma modificação no código (como uma correção de bug ou uma nova feature) para garantir que as alterações não introduziram novos defeitos em partes do sistema que já funcionavam.
- **Teste de Desempenho (Performance):** Mede a velocidade, escalabilidade e estabilidade do sistema sob condições específicas de carga.

4. Técnicas de Projeto de Testes

Para desenharmos cenários de teste eficientes sem precisar testar infinitas combinações, usamos duas técnicas consagradas:

- **Teste de Caixa Preta (Black-Box):** O analista ou desenvolvedor não conhece a estrutura interna do código. Os testes são baseados puramente nas especificações e requisitos (Entradas vs. Saídas esperadas). É muito usado em testes funcionais, de sistema e de aceitação.
- **Teste de Caixa Branca (White-Box):** O desenvolvedor tem acesso total ao código-fonte e estrutura interna do sistema. Os cenários de teste são criados para exercitar caminhos lógicos específicos (laços

de repetição, condicionais `if/else`, tratamento de exceções). É a técnica padrão para a escrita de testes de unidade.

Exercícios de Fixação — Dia 2

1. Um desenvolvedor Full Stack escreveu uma função que calcula o frete de uma entrega. No código, ele digitou um valor de taxa incorreto. Um usuário tentou comprar, o site apresentou o valor errado e a compra foi cancelada. Identifique detalhadamente onde ocorreu o **Erro**, o **Defeito** e a **Falha** neste cenário.
2. Se quisermos testar se a nossa aplicação Node.js está conseguindo enviar payloads de forma correta para a API de pagamento da Stripe, em qual **nível de teste** estamos operando? Justifique.
3. Qual a principal diferença entre a abordagem de técnica de **Caixa Preta** e **Caixa Branca**? Dê um exemplo prático de aplicação para cada uma delas.

Capítulo 3 — Testes de Unidade e Integração na Prática

1. Testes de Unidade: Objetivos e Benefícios

O teste de unidade (ou unitário) consiste em isolar a menor parte testável de um sistema (geralmente uma função ou método) de todas as suas dependências externas (como bancos de dados, APIs de terceiros ou sistemas de arquivos) e validar se o seu comportamento interno está correto.

Objetivos Principais:

- **Isolamento:** Garantir que se o teste falhar, o problema está exclusivamente na lógica daquela função específica.
- **Rapidez:** Testes de unidade rodam em milissegundos, permitindo que o desenvolvedor Full Stack tenha feedback instantâneo enquanto escreve o código.
- **Documentação Viva:** Um bom teste de unidade serve como exemplo técnico de como aquela função deve ser invocada e quais retornos ela gera.

Benefícios para o Desenvolvedor Full Stack:

- **Design de Código Limpo:** Código difícil de testar geralmente indica acoplamento excessivo ou funções que fazem coisas demais (violação do princípio de Responsabilidade Única - SRP). O teste de unidade força a criação de funções puras e modulares.
- **Segurança no Refactor:** Permite alterar a implementação interna de um algoritmo complexo para otimizar a performance com a certeza de que a regra de negócio não foi quebrada.

2. Ferramentas do Ecossistema de Desenvolvimento

Dependendo da stack tecnológica adotada no projeto, o desenvolvedor utilizará ferramentas e frameworks específicos para automatizar a execução das asserções:

Ecossistema / Linguagem	Framework de Teste de Unidade	Características Principais
Java	JUnit	O padrão da indústria para o ecossistema Java. Utiliza anotações robustas (<code>@Test</code> , <code>@BeforeEach</code>) para gerenciar o ciclo de vida dos testes.

Ecossistema / Linguagem	Framework de Teste de Unidade	Características Principais
C# / .NET	NUnit	Framework altamente extensível para a plataforma .NET, muito utilizado em aplicações empresariais e arquiteturas como Blazor.
Python	pytest	Ferramenta extremamente legível que permite escrever testes assertivos sem a necessidade de código boilerplate, suportando fixtures complexas.
JavaScript / Node.js	Jest / Vitest	Frameworks com excelente experiência de desenvolvimento (DX), que incluem ferramentas nativas de Mocking e relatórios de cobertura de código (<i>Code Coverage</i>).

3. Exemplo Prático de Caixa Branca (Testando Fluxos Lógicos)

Para fixar a técnica de **Caixa Branca**, vamos analisar um cenário onde testamos os caminhos lógicos (*if/else*) de uma função de cálculo de desconto com base no perfil do cliente.

O Código da Função (Exemplo em JavaScript/Node.js):

```
// calculadoraDesconto.js
function calcularDesconto(valorTotal, tipoCliente) {
  if (valorTotal <= 0) {
    throw new Error("O valor total deve ser maior que zero");
  }
  if (tipoCliente === 'VIP') {
    return valorTotal * 0.15; // 15% de desconto
  } else if (tipoCliente === 'PREMIUM') {
    return valorTotal * 0.10; // 10% de desconto
  }
  return 0; // Sem desconto para clientes normais
}
```

```
module.exports = { calcularDesconto };
```

A Suíte de Testes de Unidade (Caixa Branca):

Para garantir 100% de cobertura de código neste método, o desenvolvedor precisa criar cenários que passem por cada uma das ramificações lógicas:

```
// calculadoraDesconto.test.js
const { calcularDesconto } = require('./calculadoraDesconto');

describe('Testes de Unidade - Calculadora de Desconto', () => {

  // Caminho 1: Exceção / Erro lançado
  test('Deve lançar um erro se o valor total for menor ou igual a zero',
    () => {
      expect(() => calcularDesconto(-10, 'VIP')).toThrow("O valor total
deve ser maior que zero");
    });

  // Caminho 2: Cliente VIP
  test('Deve aplicar 15% de desconto para clientes do tipo VIP', () => {
    const resultado = calcularDesconto(100, 'VIP');
    expect(resultado).toBe(15);
  });

  // Caminho 3: Cliente PREMIUM
  test('Deve aplicar 10% de desconto para clientes do tipo PREMIUM', ()
=> {
    const resultado = calcularDesconto(100, 'PREMIUM');
    expect(resultado).toBe(10);
  });

  // Caminho 4: Fluxo padrão (Sem desconto)
```

```
test('Não deve aplicar desconto para clientes normais ou tipos  
inválidos', () => {  
    const resultado = calcularDesconto(100, 'REGULAR');  
    expect(resultado).toBe(0);  
});  
});
```

4. Mocks e Stubs: Isolando Componentes

Como fazemos quando a nossa função precisa buscar dados em um banco ou enviar um e-mail? Se permitirmos que o teste de unidade acesse o banco real, ele deixa de ser um teste de unidade e passa a ser um teste de integração (além de ficar lento e dependente de infraestrutura).

Para resolver isso, substituímos as dependências reais por objetos simulados:

- **Stub:** Um objeto que fornece respostas prontas e pré-configuradas para as chamadas feitas durante o teste.
- **Mock:** Um objeto mais avançado que registra como foi chamado (quantas vezes foi executado, quais argumentos recebeu), permitindo verificar o comportamento da interação.

Exercícios de Fixação — Dia 3

1. Analise o exemplo de código da `calculadoraDesconto.js` apresentado acima. Se adicionássemos uma nova regra de negócio: *"Clientes antigos (com mais de 2 anos de cadastro) recebem 5% de desconto"*, quais novos cenários de teste precisaríamos escrever para manter a cobertura de Caixa Branca completa?
2. Por que o uso de Mocks e Stubs é considerado indispensável para que um teste seja categorizado verdadeiramente como um **Teste de Unidade**?
3. Escolha uma das ferramentas de teste citadas na tabela (JUnit, NUnit, pytest ou Jest) e pesquise qual é a palavra-chave/anotação padrão utilizada por ela para realizar uma asserção (verificação de igualdade).

Capítulo 4 — Integração, Regressão e Validação do Sistema

1. Testes de Integração: Objetivos e Abordagens

Se o teste de unidade garante que cada peça do motor funciona isoladamente, o **Teste de Integração** é o momento em que juntamos essas peças para garantir que o motor de fato dê a partida. No desenvolvimento Full Stack, os erros de integração costumam acontecer nas interfaces entre módulos (ex: o front-end enviando um parâmetro com nome diferente do que a API espera, ou a API gerando uma consulta SQL que viola uma restrição de chave estrangeira no banco de dados).

Objetivos dos Testes de Integração:

- Verificar a comunicação entre módulos internos do sistema.
- Validar a integração com serviços externos (gateways de pagamento, serviços de e-mail, APIs de mapas).
- Garantir a integridade da persistência de dados (leitura e escrita no banco de dados).

Principais Abordagens de Integração:

- **Big Bang:** Todos os módulos são integrados de uma só vez e o sistema é testado por completo. Embora pareça simples, é uma abordagem perigosa e desencorajada, pois se um erro acontece, é extremamente difícil isolar qual módulo o causou.
- **Top-Down (De Cima para Baixo):** Começa testando os módulos de nível superior (como a camada de controle da API) e vai descendo para as camadas inferiores. Módulos que ainda não foram desenvolvidos são substituídos por *Stubs* temporários.
- **Bottom-Up (De Baixo para Cima):** Começa testando os módulos mais baixos e estruturais (como os repositórios de acesso ao banco de dados) e vai subindo até as interfaces de usuário. Módulos superiores que faltam são simulados por drivers de teste.

2. Testes de Regressão: Blindando o Código Existente

Um dos maiores medos de um time de desenvolvimento é o famoso *"Consertei um bug aqui e quebrei três funcionalidades que já estavam funcionando ali"*. É para mitigar esse risco que existem os **Testes de Regressão**.

O que são e qual a sua importância?

O teste de regressão consiste em reexecutar uma seleção de testes funcionais e de unidade após qualquer modificação no código-fonte (seja para adicionar uma nova funcionalidade, realizar uma refatoração ou aplicar uma correção urgente).

A automação desempenha um papel crítico aqui. Como a suíte de regressão cresce proporcionalmente ao tamanho do software, executá-la manualmente a cada alteração tornaria o ciclo de entrega lento e propenso a

falhas humanas. Automatizar esse processo garante que o time possa lançar novas versões em produção várias vezes ao dia com total segurança.

3. Testes de Sistema: A Visão de Ponta a Ponta (End-to-End)

O **Teste de Sistema** eleva o nível de validação para o produto completo e totalmente integrado. Neste nível, a aplicação é executada em um ambiente que replica fielmente o cenário real de produção (homologação).

```
+-----+HTTP+-----+SQL+-----+
|Front-end| ----->|API|----->|Banco de Dados|
|(Interface)|<-----|(Rotas)|<-----|(Persistência)|
+-----+JSON/REST+-----+ +-----+
```

Diferente do teste de unidade, onde simulamos o banco de dados com um Mock, no Teste de Sistema o fluxo roda de forma real. Um script automatizado ou um testador abre o navegador web, preenche um formulário na tela (Front-end), clica em enviar, a requisição trafega pela rede, atinge o servidor web, dispara a lógica de negócio na API, persiste a informação no banco de dados e retorna o sucesso visual na tela do usuário.

- **Objetivo:** Validar o comportamento do sistema como um todo frente aos requisitos de negócio e técnicos estipulados.
- **Ferramentas comuns para o ambiente:** Docker (para subir réplicas de bancos e serviços), Postman/Insomnia (para testes de rotas integradas) e frameworks de automação E2E.

Exercícios de Fixação — Dia 4

1. Explique por que a abordagem de testes do tipo **Big Bang** costuma gerar um alto custo de depuração (*debugging*) quando um defeito é encontrado, em comparação com abordagens gradativas.
2. Imagine que sua equipe acabou de subir uma correção de bug em produção para resolver um erro na tela de login, mas logo em seguida os usuários relataram que o botão de "Esqueci minha senha" parou de funcionar. Que tipo de teste falhou em prevenir essa situação no ambiente de desenvolvimento? Justifique.
3. Qual é a principal diferença conceitual entre o comportamento de uma dependência (como um banco de dados) em um **Teste de Unidade** e em um **Teste de Sistema**?

Capítulo 5 — Automação de Testes, Experiência do Usuário e CI/CD

1. Introdução à Automação de Testes: Benefícios e Desafios

À medida que os sistemas crescem, realizar testes puramente manuais torna-se inviável. A automação de testes consiste no uso de ferramentas de software para controlar a execução de testes, comparando os resultados reais com os resultados esperados de forma programática.

- **Benefícios:**
 - **Velocidade e Execução Repetitiva:** Testes automatizados podem rodar centenas de vezes por dia (por exemplo, a cada alteração no repositório Git).
 - **Consistência:** Elimina o risco de falha humana por cansaço ou distração durante rotinas repetitivas de testes de regressão.
 - **Feedback Rápido:** Permite que o desenvolvedor saiba em poucos minutos se o novo código introduziu algum defeito.
- **Desafios:**
 - **Custo Inicial Alto:** Escrever scripts de teste exige tempo de engenharia e planejamento de arquitetura.
 - **Manutenção de Código:** Se a interface visual ou as regras de negócio mudarem drasticamente, os testes automatizados precisam ser atualizados, gerando esforço contínuo de manutenção.

2. Testes de Interface de Usuário (UI) e Ferramentas do Mercado

Os **Testes de Interface de Usuário (UI)** simulam as interações reais de um utilizador no ecrã (clicar em botões, preencher formulários, rolar páginas e verificar se os elementos visuais estão renderizados corretamente).

No ecossistema de desenvolvimento, destacam-se três grandes ferramentas:

- **Cypress:** Muito popular no desenvolvimento Web moderno. Executa diretamente dentro do navegador, sendo extremamente rápido, fácil de configurar e excelente para testar aplicações Single Page Applications (SPAs) feitas em React, Vue ou Angular.
- **Selenium:** A ferramenta pioneira do mercado. Suporta quase todas as linguagens de programação (Java, C#, Python, JavaScript) e é ideal para testes multiplataforma complexos em diferentes navegadores (Cross-Browser).
- **Appium:** Uma extensão do conceito do Selenium voltada especificamente para cenários móveis, permitindo automatizar aplicações nativas e híbridas para Android e iOS.

3. Testes de API: Garantindo o Contrato de Dados

Enquanto o teste de UI foca na camada visual, o **Teste de API** valida as regras de negócio expostas via endpoints HTTP (REST ou GraphQL) sem passar pela interface gráfica.

Ele verifica:

- Se os códigos de status HTTP estão corretos (ex: **200 OK**, **201 Created**, **400 Bad Request**).
- Se o tempo de resposta da rota está dentro do esperado.
- Se a estrutura e o conteúdo do payload JSON retornado respeitam o contrato estabelecido entre o Front-end e o Back-end.

Exemplo Prático de Script de Teste de API (Exemplo Conceitual com Cypress):

```
describe('Testes Automatizados de API - Módulo de Produtos', () => {  
  it('Deve validar a listagem de produtos com sucesso', () => {  
    cy.request('GET', 'http://api.meusistema.com/v1/produtos')  
      .then((response) => {  
        expect(response.status).to.eq(200);  
        expect(response.body).to.be.an('array');  
        expect(response.body[0]).to.have.property('nome');  
      });  
  });  
});
```

4. Testes de Aceitação, Usabilidade e a Integração Contínua (CI/CD)

Para fechar o ciclo de desenvolvimento, o software precisa ser validado sob a ótica do cliente final e integrado ao pipeline automatizado de publicação:

- **Testes de Aceitação e Usabilidade:** Avaliam se o fluxo lógico faz sentido para o utilizador e se atende aos critérios de aceitação do negócio. O cliente ou o Product Owner (PO) participa ativamente aqui, validando se a experiência de utilização (UX) é fluida e intuitiva.
- **Integração Contínua e Entrega Contínua (CI/CD):** Num fluxo de trabalho moderno, os testes automatizados criados nos Dias 3, 4 e 5 são colocados numa "esteira automatizada" (pipeline). Sempre que um desenvolvedor faz um **git push** com uma nova funcionalidade, o servidor de CI (como GitHub Actions ou GitLab CI) intercepta o código, baixa as dependências, roda toda a suíte de testes (Unidade,

Integração, API e UI). Se qualquer teste falhar, o build é rejeitado e o deploy para produção é bloqueado imediatamente, mantendo o ambiente seguro e estável.

Exercícios de Fixação — Dia 5

1. Explique o conceito de "Falso Positivo" ou "Falso Negativo" no contexto da automação de testes e como a necessidade de manutenção constante dos scripts se relaciona com isso.
2. Numa arquitetura Full Stack separada (onde o Front-end consome uma API de forma independente), por que os testes de API são considerados mais rápidos e estáveis do que os testes de Interface de Usuário (UI)?
3. Desenhe ou descreva textualmente as etapas pelas quais o código de um desenvolvedor passa num pipeline de CI/CD, desde o momento do `git push` até à disponibilização final em ambiente de Produção, indicando em que momento os testes automatizados devem ser disparados.

REFERÊNCIAS

1. **AMBLER**, Scott. **Refatoração de Bancos de Dados**: Evolução Evolucionária de Design. 1. ed. Porto Alegre: Bookman, 2007.
2. **ANICETO**, Maurício. **Testes automatizados de software**: Um guia prático. 1. ed. São Paulo: Casa do Código, 2018.
3. **INTERNATIONAL ORGANIZATION FOR STANDARDIZATION**. **ISO/IEC 25010**: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models. 2011. Disponível em: <https://www.iso.org/standard/35733.html>. Acesso em: 03 jul. 2026.
4. **MOLINARI**, Leonardo. **Testes de Software**: Produzindo Sistemas de Alta Qualidade. 4. ed. Rio de Janeiro: LTC, 2022.
5. **MYERS**, Glenford J. **The Art of Software Testing**. 3. ed. Hoboken: John Wiley & Sons, 2011.
6. **PRESSMAN**, Roger S.; **MAXIM**, Bruce R. **Engenharia de Software**: Uma Abordagem Profissional. 9. ed. Porto Alegre: AMGH, 2021.
7. **SOMMERVILLE**, Ian. **Engenharia de Software**. 10. ed. São Paulo: Pearson Education do Brasil, 2019.